LECTURE 18

STRUCTURING REASONING IN LLMS, WITH LOGIC

DATASCI290: NEUROSYMBOLIC AI, SPRING 2026

MARCH 11 2026

REFERENCES

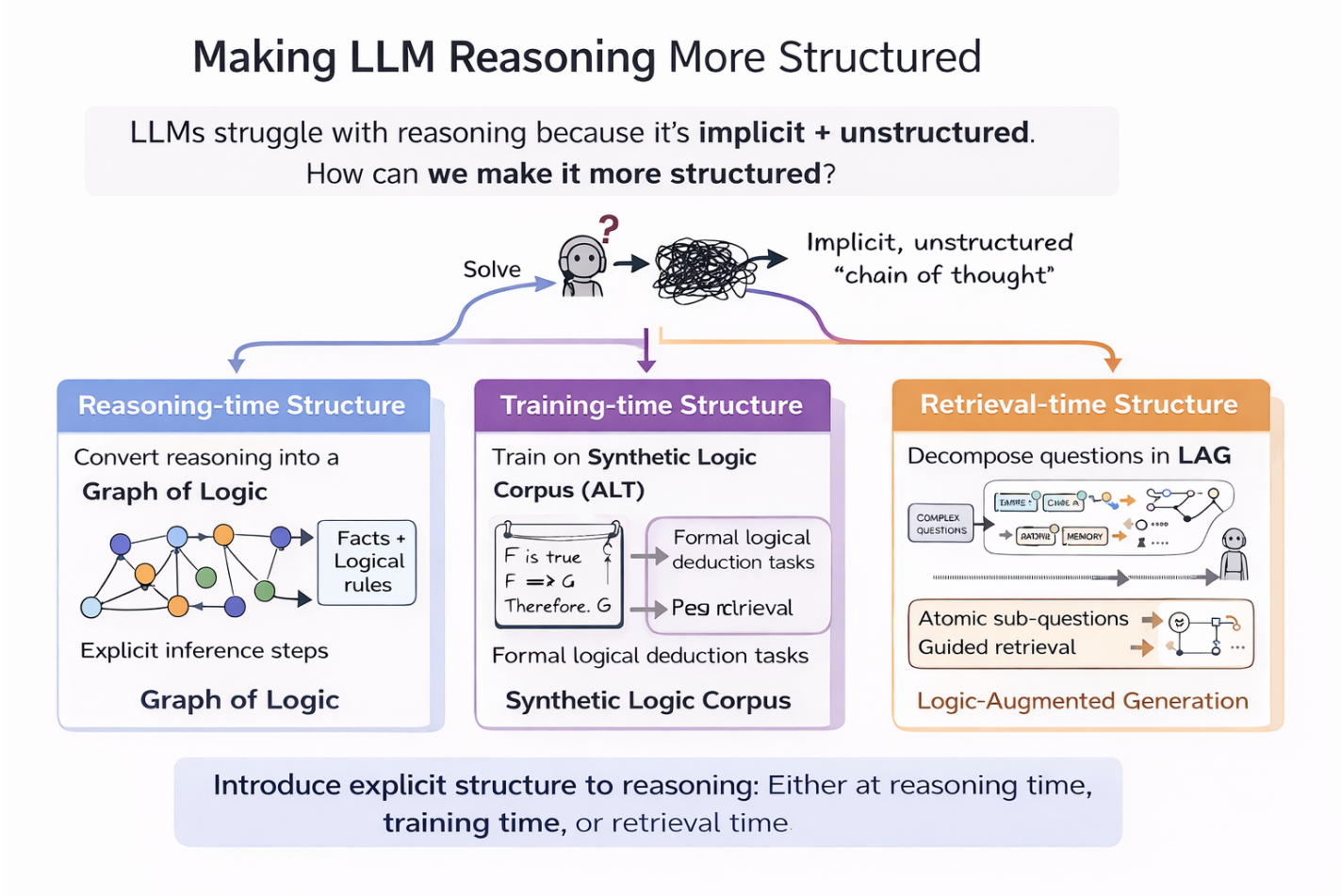
- Alotaibi, F., Kulkarni, A., & Zhou, D. (2024, December). Graph of logic: Enhancing llm reasoning with graphs and symbolic logic. In *2024 IEEE International Conference on Big Data (BigData)* (pp. 5926-5935). IEEE.
- Morishita, T., Morio, G., Yamaguchi, A., & Sogawa, Y. (2024). Enhancing reasoning capabilities of llms via principled synthetic logic corpus. *Advances in Neural Information Processing Systems*, *37*, 73572-73604.
- Xiao, Y., Zhou, C., Zhang, Y., Zhang, Q., Dong, S., Chen, S., Yang, C. and Huang, X., 2025. Lag: Logic-augmented generation from a cartesian perspective. *arXiv preprint arXiv:2508.05509*.
- Pan, L., Albalak, A., Wang, X., & Wang, W. (2023, December). Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 3806-3824).

MAKING LLM REASONING **STRUCTURED**

- Large language models produce reasoning as a side-effect of next-token prediction rather than from explicit rules.
- This lecture examines three research directions that introduce structure into LLM reasoning pipelines.
- Let's look at different paradigms:
  - "Soft" logic integration
    - Graph of Logic (structured inference graphs),
    - Additional Logic Training (synthetic logic training data)
    - Logic-Augmented Generation (logic-guided retrieval).
  - "Hard" logic integration
    - Symbolic solver integration
- Hypothesis: reasoning reliability improves when intermediate reasoning states become explicit objects rather than hidden text.

SOFT INTEGRATION

STRUCTURED REASONING



WHY LLM REASONING OFTEN FAILS

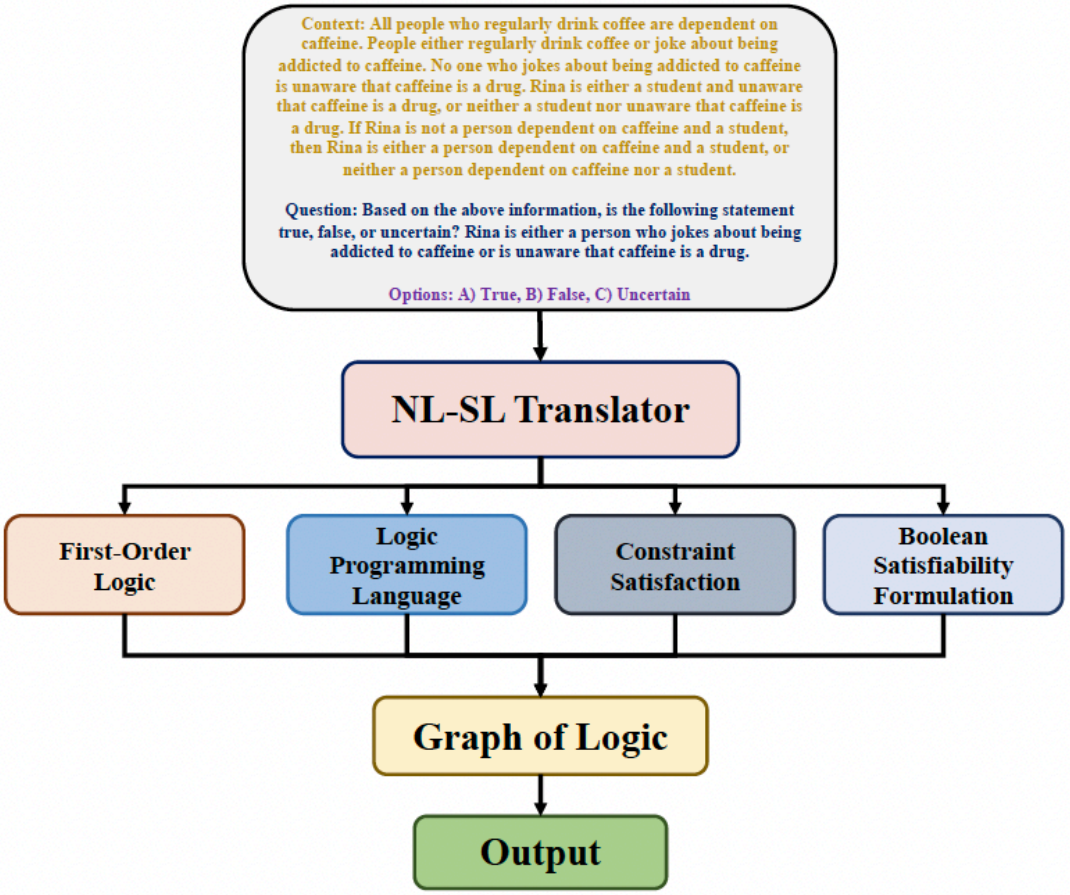
- LLMs approximate $P(\text{token} \mid \text{context})$
  - which captures correlations rather than deductive rules
- As reasoning chains grow longer, small token errors propagate and cause incorrect conclusions.
- Common failure modes, when the problem requires
  - multi-step inference
  - compositional reasoning
  - unseen entities.
- Can explicit reasoning **structures** (that constrain or guide LLM inference) help ?

THREE STRUCTURAL ENTRY POINTS

- Approaches
  - **Inference-time** reasoning structure:
    - represent reasoning steps as graph objects that can be checked (Graph of Logic).
  - **Training-time** structure
    - train the model on explicit logical deduction examples (ALT).
  - **Retrieval-time** structure: decompose complex questions into atomic reasoning steps (LAG).
- These approaches modify different parts of the LLM pipeline
  - But share the goal of stabilizing reasoning

GRAPH OF LOGIC (INFERENCE TIME)

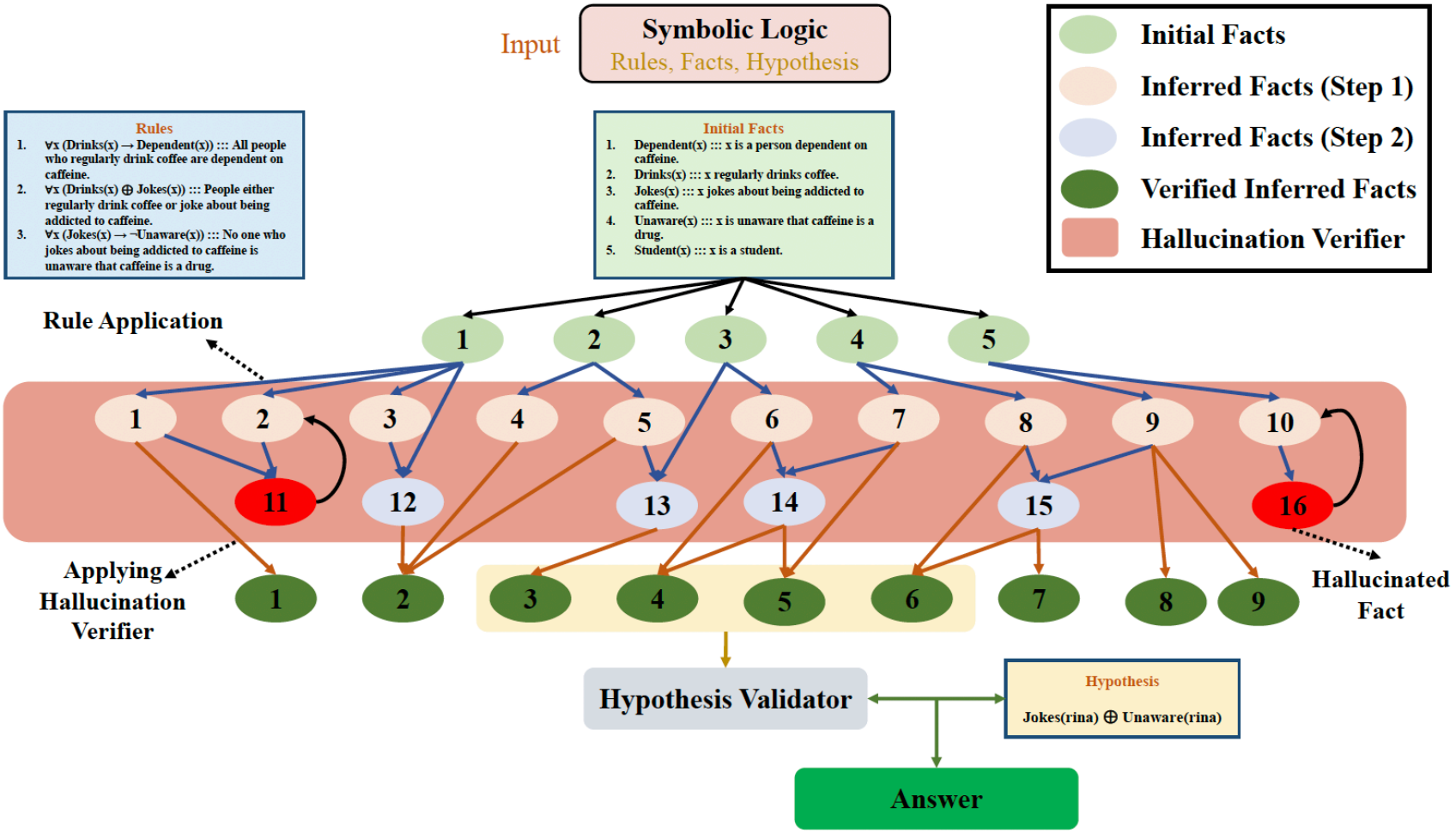
- Instead of letting the LLM generate a free-form chain of thought, reasoning is represented as a structured graph.
- Nodes correspond to propositions or facts.
- Edges correspond to logical inference rules used to derive new facts.
- The LLM proposes candidate reasoning steps which are then checked for logical consistency.



FORMAL REPRESENTATION IN(THE) **GRAPH OF LOGIC**

- The reasoning structure is represented as a graph G = (V, E).
- V: set of propositions including initial facts and inferred conclusions.
- E: inference operations linking premises to derived statements.
- *Graph expansion* corresponds to applying reasoning rules to derive new propositions.
- This creates an *explicit reasoning trace* that can be inspected or validated.

SYSTEM


Input
Symbolic Logic
Rules, Facts, Hypothesis
Initial Facts
Inferred Facts (Step 1)
Inferred Facts (Step 2)
Verified Inferred Facts
Hallucination Verifier
Rules
1. ∀x (Drinks(x) → Dependent(x)) ::: All people who regularly drink coffee are dependent on caffeine.
2. ∀x (Drinks(x) ⊕ Jokes(x)) ::: People either regularly drink coffee or joke about being addicted to caffeine.
3. ∀x (Jokes(x) → ¬Unaware(x)) ::: No one who jokes about being addicted to caffeine is unaware that caffeine is a drug.
Initial Facts
1. Dependent(x) ::: x is a person dependent on caffeine.
2. Drinks(x) ::: x regularly drinks coffee.
3. Jokes(x) ::: x jokes about being addicted to caffeine.
4. Unaware(x) ::: x is unaware that caffeine is a drug.
5. Student(x) ::: x is a student.
Rule Application
Applying Hallucination Verifier
Hallucinated Fact
Hypothesis Validator
Hypothesis
Jokes(rina) ⊕ Unaware(rina)
Answer

GRAPH EXPANSION PROCESS

- Initial nodes correspond to known premises extracted from the problem.
- The LLM proposes candidate logical deductions based on these premises.
- Each proposed deduction becomes a candidate new node in the graph.
- Edges record which premises and rules were used to derive the new node.

LOGICAL **VERIFICATION** STEP

- Not all generated reasoning steps are valid !
- The system applies logical consistency checks to verify each candidate deduction.
- If a proposed fact *contradicts existing logical constraints* it is **discarded**
- This step acts as **a hallucination filter** for reasoning steps.

HYPOTHESIS TESTING IN GRAPH OF LOGIC

- Final goal: test whether a target hypothesis logically follows from the graph.
- Only facts that pass logical verification are considered.
- Reasoning graph: provides a structured explanation of how the conclusion was reached.
- This improves interpretability
  - Compared with free-form chain-of-thought reasoning.

PROMPTS

New Fact Inference

<Instructions>
Given an initial fact and a set of logical rules, derive all possible inferred facts. Apply each rule iteratively until no more new conclusions can be drawn. Provide the inferred facts in logical format, along with their descriptions.
</Instructions>
<Approach>
1. Start with the initial fact.
2. Apply each rule to the current set of facts.
3. If a new fact is derived, add it to the set of known facts.
4. Repeat until no new facts can be derived.
5. Present the final set of inferred facts in logical format with descriptions.
</Approach>
<Example>
Initial Fact:
Dependent(x) ::: x is a person dependent on caffeine.
Rules:
∀x (Drinks(x) → Dependent(x)) ::: All people who regularly drink coffee are dependent on caffeine.
∀x (Drinks(x) ⊕ Jokes(x)) ::: People either regularly drink coffee or joke about being addicted to caffeine.
∀x (Jokes(x) → ¬Unaware(x)) ::: No one who jokes about being addicted to caffeine is unaware that caffeine is a drug.
Output:
Dependent(rina) ::: Rina is a person dependent on caffeine.
Drinks(rina) ⊕ Jokes(rina) ::: Rina either regularly drinks coffee or jokes about being addicted to caffeine.
Jokes(rina) → ¬Unaware(rina) ::: If Rina jokes about being addicted to caffeine, then she is not unaware that caffeine is a drug.
</Example>

Hallucination Verifier

<Instructions>
Conduct resolution refutation on the following set of inferred facts and rules. Please score the resolution refutation in terms of how consistent the inferred facts are with the given rules. A score of 10 implies that each inferred fact is inconsistent with the given rules, while a score of 0 implies that at least half of the inferred facts are inconsistent with the given rules. You may provide reasoning for your scoring, but the final score for consistency should be between the tags <Consistency> and </Consistency>, without any additional text within the tag.
</Instructions>
<Approach>
1. Negate the Inferred Facts.
2. Convert All Statements into CNF.
3. Apply Resolution.
4. Derive Contradiction.
</Approach>
<Example>
Inferred Facts:
Dependent(rina) ::: Rina is a person dependent on caffeine.
Drinks(rina) ⊕ Jokes(rina) ::: Rina either regularly drinks coffee or jokes about being addicted to caffeine.
Jokes(rina) → ¬Unaware(rina) ::: If Rina jokes about being addicted to caffeine, then she is not unaware that caffeine is a drug.
Rules:
∀x (Drinks(x) → Dependent(x)) ::: All people who regularly drink coffee are dependent on caffeine.
∀x (Drinks(x) ⊕ Jokes(x)) ::: People either regularly drink coffee or joke about being addicted to caffeine.
∀x (Jokes(x) → ¬Unaware(x)) ::: No one who jokes about being addicted to caffeine is unaware that caffeine is a drug.
Resolution Refutation:
Negate these inferred facts:
1. ¬Dependent(rina)
2. ¬(Drinks(rina) ⊕ Jokes(rina))
3. ¬(Jokes(rina) → ¬Unaware(rina))
Convert all statements into CNF:
1. ∀x (Drinks(x) → Dependent(x)) ::: ¬Drinks(x) ∨ Dependent(x)
2. ∀x (Drinks(x) ⊕ Jokes(x)) ::: (Drinks(x) ∨ Jokes(x)) ∧ (¬Drinks(x) ∨ ¬Jokes(x))
3. ∀x (Jokes(x) → ¬Unaware(x)) ::: ¬Jokes(x) ∨ ¬Unaware(x)
Apply resolution:
Given ¬Dependent(rina) and ¬Drinks(rina) ∨ Dependent(rina), we can't directly resolve.
Given (¬Drinks(rina) ∧ ¬Jokes(rina)) ∨ (Drinks(rina) ∧ Jokes(rina)), this means either (¬Drinks(rina) ∧ ¬Jokes(rina)) or (Drinks(rina) ∧ Jokes(rina)).
Given Jokes(rina) ∧ Unaware(rina).
From the original rules:
¬Jokes(rina) ∨ ¬Unaware(rina)
We resolve Jokes(rina) ∧ Unaware(rina) with ¬Jokes(rina) ∨ ¬Unaware(rina) to derive a contradiction.
Conclusion:
We derived a contradiction from the negation of the inferred facts with the given rules, so the original inferred facts are consistent with the rules.
Output: <Consistency>10</Consistency>
</Example>

Hypothesis Validator

<Instructions>
Given a problem statement as contexts, the task is to answer a logical reasoning question. Use the inferred facts to help you derive the conclusion. Output only the letter corresponding to the chosen option.
</Instructions>
<Approach>
1. Read and understand the prompt. Identify the main question.
2. Identify key concepts or terms.
3. Use the aggregated facts to draw your conclusion.
</Approach>
<Example>
Context: All people who regularly drink coffee are dependent on caffeine. People either regularly drink coffee or joke about being addicted to caffeine. No one who jokes about being addicted to caffeine is unaware that caffeine is a drug. Rina is either a student and unaware that caffeine is a drug, or neither a student nor unaware that caffeine is a drug. If Rina is not a person dependent on caffeine and a student, then Rina is either a person dependent on caffeine and a student, or neither a person dependent on caffeine nor a student.

Question: Based on the above information, is the following statement true, false, or uncertain? Rina is either a person who jokes about being addicted to caffeine or is unaware that caffeine is a drug.

Options: A) True, B) False, C) Uncertain

Aggregated Facts:
Dependent(rina) ::: Rina is a person dependent on caffeine.
Drinks(rina) ⊕ Jokes(rina) ::: Rina either regularly drinks coffee or jokes about being addicted to caffeine.
Jokes(rina) → ¬Unaware(rina) ::: If Rina jokes about being addicted to caffeine, then she is not unaware that caffeine is a drug.
Student(rina) ∧ Unaware(rina) ::: Rina is a student and unaware that caffeine is a drug.
Analysis:
- From (Student(rina) ∧ Unaware(rina)) ⊕ ¬(Student(rina) ∨ Unaware(rina)), Rina can be either (Student(rina) ∧ Unaware(rina)) or ¬(Student(rina)) ∧ ¬(Unaware(rina)).
- If Jokes(rina) is true, ¬Unaware(rina) must be true.
Conclusion:
- When ¬(Student(rina)) ∧ ¬(Unaware(rina)), Jokes(rina) ⊕ Unaware(rina) depends on Jokes(rina).
- When (Student(rina) ∧ Unaware(rina)), Jokes(rina) ⊕ Unaware(rina) is true.
- "Jokes(rina) ⊕ Unaware(rina)" can be true or false, so it is Uncertain.

Output: C
</Example>

EVALUATION

- **FOLIO** : natural-language first-order logic reasoning; decide whether a conclusion is entailed, contradicted, or unknown.
- **LogicalDeduction** : ordering and arrangement puzzles; infer positions or relative order from a small set of constraints.
- **ProofWriter** : multi-step deductive reasoning from facts and rules written in natural language.
- **PrOntoQA** : synthetic benchmark for controlled multi-hop logical deduction, designed to test whether models truly follow logical chains.
- **AR-LSAT** : analytical reasoning questions from the LSAT; hard constraint-based logic games.
- **CLUTRR** : relational reasoning about family or social relationships; infer unseen relations from chains of facts.
- **AbductiveRules :** abductive reasoning benchmark; infer the most plausible missing cause or rule behind observed facts.

TABLE I
DATASET STATISTICS

Dataset	In-context Size	Test Size
FOLIO	5	204
Logical Deduction	5	300
PrOntoQA	5	500
ProofWriter	5	600
AR-LSAT	5	231
CLUTRR	5	100
AbductiveRules	5	100

RESULTS

TABLE II
RESULTS COMPARING LLM REASONING CAPABILITIES ON DIFFERENT REASONING DATASETS USING DIFFERENT APPROACHES.

Dataset	Reasoning Type	Model	I/O	CoT	ToT	GoT	GoL
PrOntoQA	Deductive	GPT-3.5	47.40	67.80	68.00	40.40	**73.00**
		GPT-4	77.40	**98.79**	48.00	67.20	80.40
ProofWriter	Deductive	GPT-3.5	35.50	49.17	56.31	40.33	**63.43**
		GPT-4	52.67	68.11	38.60	69.33	**76.90**
FOLIO	Deductive	GPT-3.5	45.09	57.35	35.20	31.31	**76.10**
		GPT-4	69.11	76.58	51.40	64.40	**77.11**
Logical Deduction	Deductive	GPT-3.5	40.00	42.23	21.00	37.31	**46.28**
		GPT-4	71.33	75.75	43.33	43.00	**76.09**
AR-LSAT	Deductive	GPT-3.5	20.39	17.31	19.70	31.70	**35.20**
		GPT-4	33.33	35.66	23.90	31.70	**45.00**
CLUTRR	Inductive	GPT-3.5	45.09	57.35	35.20	31.31	**60.00**
		GPT-4	78.00	85.00	40.00	58.00	**87.00**
AbductiveRules	Abductive	GPT-3.5	30.00	60.00	74.00	66.00	**85.00**
		GPT-4	40.00	**80.00**	70.00	63.00	79.00

TABLE III
RESULTS COMPARING OUR PROPOSED APPROACH WITH RECENT APPROACH [1] THAT USES SYMBOLIC SOLVERS INSTEAD OF LLMS FOR INFERENCE.

Dataset	Model	Logic-LM	GoL
PrOntoQA	GPT-3.5	61.00	**73.00**
	GPT-4	**83.20**	80.40
ProofWriter	GPT-3.5	**71.45**	63.43
	GPT-4	**79.66**	76.90
FOLIO	GPT-3.5	61.27	**76.10**
	GPT-4	**78.92**	77.11
Logical Deduction	GPT-3.5	**65.67**	46.28
	GPT-4	**87.63**	76.09
AR-LSAT	GPT-3.5	26.41	**35.20**
	GPT-4	43.90	**45.00**

TABLE IV
RESULTS DEMONSTRATING THE PERFORMANCE OF HALLUCINATION DETECTOR IN DETECTING HALLUCINATED FACTS.

Dataset	Hallucination Detector Accuracy
PrOntoQA	88.00
ProofWriter	70.00
Folio	75.80
Logical Deduction	57.00
AR-LSAT	68.80
CLUTRR	75.00
AbductiveRules	68.00

- Evaluated on multiple reasoning benchmarks.
- Compared with Chain-of-Thought, Tree-of-Thought, and other prompting strategies.
- Structured reasoning graphs improve accuracy and reduce inconsistent reasoning.
- The major benefit arises from filtering invalid intermediate reasoning steps.

ADDITIONAL LOGIC **TRAINING**

ADDITIONAL LOGIC TRAINING (ALT): MOTIVATION

- Many LLM training corpora contain **very little explicit logical deduction**
  - Therefore models may never learn general logical inference patterns
- ALT proposes explicitly teaching reasoning through **synthetic logic datasets**.
- The goal is to improve reasoning ability by exposing models to many structured deduction tasks.

SYNTHETIC LOGIC DATASET CONSTRUCTION

- Dataset used: FLD_{x2} :(Formal Logic Deduction)
  - generated programmatically.
- Each example contains a set of premises, logical rules, and a hypothesis.
- The model must determine whether the hypothesis follows from the premises.
- Problems are designed to require multiple reasoning steps.

WHY SYNTHETIC ENTITIES ARE USED

- Entities in the dataset often use arbitrary symbols or invented names.
- This prevents the model from relying on world knowledge memorization.
- The model must rely purely on logical relationships between statements.
- This forces the learning of abstract reasoning patterns.

EXAMPLE LOGICAL DEDUCTION PATTERN

- Consider the classical rule of modus ponens.
- If statement F is true and F → G is true, then G must also be true.
- Synthetic datasets generate many such logical patterns with different linguistic forms.
- The model learns to recognize the underlying logical structure across variations.

DESIGN PRINCIPLES OF THE DATASET

- Include both valid and invalid reasoning chains.
- Provide diverse rule types such as implication, negation, conjunction.
- Construct multi-step deduction chains requiring sequential reasoning.
- Express the same rule in multiple natural language formulations.

TRAINING PROCEDURE

- Large language models are **fine-tuned** on the synthetic logic dataset.
- The objective is standard language modeling but applied to reasoning tasks.
- Through exposure to thousands of deduction problems the model internalizes reasoning patterns.
- This training stage is referred to as Additional Logic Training (ALT).

ALT: KEY EXPERIMENTAL RESULTS

- Evaluation Datasets
  - **BAbI**:synthetic tasks testing basic reasoning skills such as deduction, induction, and multi-step inference.
  - **FOLIO**:natural language **first-order logic** reasoning (entailment, contradiction, unknown).
  - **LogicNLI**:logical natural language inference benchmark with formal reasoning patterns.
  - **RobustLR**:adversarial logical reasoning tasks designed to test robustness of logical inference.
  - **AR-LSAT**:analytical reasoning questions from the **LSAT logic games** section.
  - **LogiQA**:reading comprehension questions requiring **formal logical reasoning**.
  - **ReClor**:logical reasoning benchmark based on **standardized exam questions**.
  - **AbductionRules**:abductive reasoning tasks requiring inference of the **best explanation**.
  - **ART**:abductive reasoning challenge (choose the most plausible hypothesis)
- After logic training, models show major improvements on logical reasoning benchmarks.
- Improvements up to roughly 30 percentage points are reported.
- Gains also transfer partially to mathematical reasoning and natural language inference tasks.
- This suggests reasoning ability can be improved by targeted training data.

Table 4: Benchmark-wise 5-shot performance of LLaMA-3.1-70B before and **after** ALT on $FLD_{\times 2}$. Refer to Table F.9 for LLaMA-3.1-8B results. Table E.7 details each benchmark.

(a) Logic.

	bAbiD	FOLIO	LogicNLI	RobustLR	AR-LSAT	LogiQA	ReClor	AbductionR	ART
LLaMA-3.1-70B	**83.8**$_{\pm 1.2}$	58.9$_{\pm 1.6}$	34.9$_{\pm 1.1}$	49.6$_{\pm 0.9}$	21.5$_{\pm 1.0}$	64.3$_{\pm 1.2}$	33.7$_{\pm 0.7}$	84.0$_{\pm 0.7}$	85.4$_{\pm 0.9}$
⊕**ALT-FLD**$_{\times 2}$	83.5$_{\pm 0.5}$	**66.7**$_{\pm 0.6}$	**50.9**$_{\pm 0.5}$	**81.6**$_{\pm 0.3}$	**25.0**$_{\pm 0.4}$	**69.4**$_{\pm 0.5}$	**36.3**$_{\pm 0.3}$	**95.7**$_{\pm 0.2}$	**85.5**$_{\pm 0.4}$

(b) Math.

	GSM8k		MATH	MathQA
	CoT	CoT (0-shot)	-	-
LLaMA-3.1-70B	80.9$_{\pm 1.1}$	75.2$_{\pm 1.2}$	65.4$_{\pm 1.3}$	23.7$_{\pm 0.6}$	55.0$_{\pm 0.9}$
⊕**ALT-FLD**$_{\times 2}$	**83.3**$_{\pm 0.4}$	**80.4**$_{\pm 0.4}$	**73.0**$_{\pm 0.5}$	**24.4**$_{\pm 0.2}$	**55.4**$_{\pm 0.4}$

(c) Code.

	HumanEval	MBPP	MBPP+	MultiPL-E (cpp)	MultiPL-E (go)
LLaMA-3.1-70B	32.3	43.4	48.7	29.8	76.6
⊕**ALT-FLD**$_{\times 2}$	**42.6**	**49.5**	**52.5**	**38.7**	**78.6**

(d) Natural language inference (NLI).

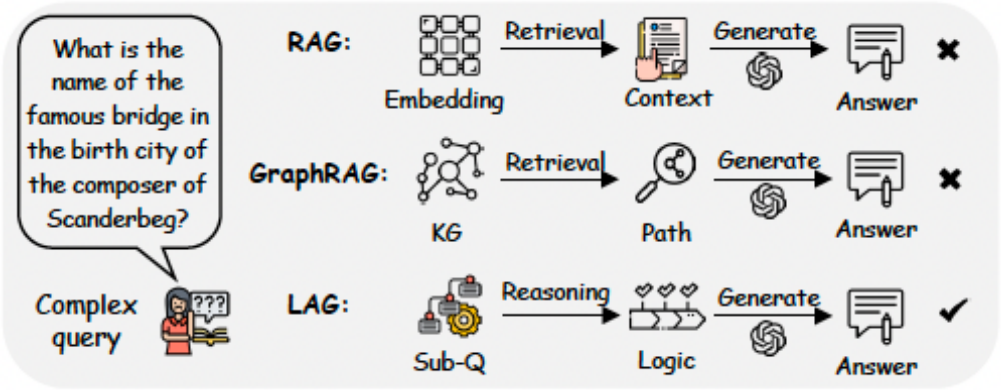
	HELP	MNLI	RTE	SNLI
LLaMA-3.1-70B	45.8$_{\pm 0.5}$	82.2$_{\pm 0.4}$	84.0$_{\pm 0.7}$	**82.6**$_{\pm 0.4}$
⊕**ALT-FLD**$_{\times 2}$	**51.3**$_{\pm 0.2}$	**83.7**$_{\pm 0.2}$	**87.2**$_{\pm 0.3}$	82.3$_{\pm 0.2}$

(e) Others.

	CommonsenseQA	HellaSwag	SQuAD	WinoGrande	ARCe	ARCc	GPQA	OpenBookQA	SciQ
LLaMA-3.1-70B	81.2$_{\pm 1.1}$	69.2$_{\pm 0.5}$	38.5$_{\pm 0.0}$	85.6$_{\pm 1.0}$	89.1$_{\pm 0.6}$	65.3$_{\pm 1.4}$	**40.7**$_{\pm 1.4}$	41.4$_{\pm 0.7}$	**98.5**$_{\pm 0.4}$
⊕**ALT-FLD**$_{\times 2}$	**82.5**$_{\pm 0.4}$	**69.6**$_{\pm 0.2}$	**40.1**$_{\pm 0.0}$	**86.1**$_{\pm 0.4}$	**89.4**$_{\pm 0.3}$	**66.7**$_{\pm 0.6}$	40.6$_{\pm 0.6}$	**42.8**$_{\pm 0.3}$	**98.5**$_{\pm 0.2}$

LAG:
LOGIC AUGMENTED GENERATION

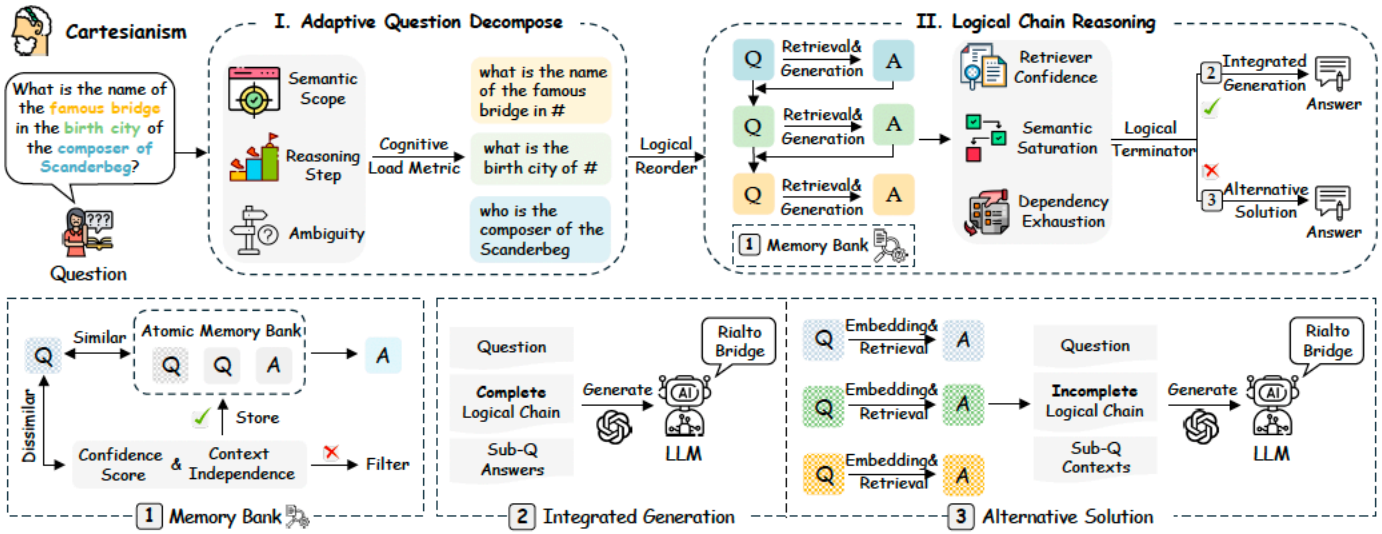
LOGIC-AUGMENTED GENERATION (LAG): MOTIVATION



- Traditional retrieval-augmented generation retrieves documents based on semantic similarity.
- Complex questions often require multi-hop reasoning across several facts.
- A single retrieval step cannot capture the logical dependencies of such questions.
- LAG introduces **structured reasoning before retrieval**.

DESCARTES' FRAMEWORK

- Descartes (in *Discours de la méthode*) proposed **four** principles for solving problems scientifically:
  - **Doubt** everything, avoiding precipitancy and prejudice
  - **Divide** any complex question into multiple simpler sub-questions
  - **Order** sub-questions, from the simplest to the most complex and fix them step by step
  - Once all issues are solved, **review** them to ensure that nothing was omitted.


Cartesianism
What is the name of the famous bridge in the birth city of the composer of Scanderbeg?
Question
I. Adaptive Question Decompose
Semantic Scope
Reasoning Step
Ambiguity
Cognitive Load Metric
what is the name of the famous bridge in #
what is the birth city of #
who is the composer of the Scanderbeg
Logical Reorder
II. Logical Chain Reasoning
Q
Retrieval& Generation
A
1 Memory Bank
Retriever Confidence
Semantic Saturation
Dependency Exhaustion
Logical Terminator
2 Integrated Generation
Answer
3 Alternative Solution
Answer
Similar
Atomic Memory Bank
Store
Dissimilar
Confidence Score
&
Context Independence
Filter
1 Memory Bank
Question
Complete Logical Chain
Sub-Q Answers
Generate
LLM
Rialto Bridge
2 Integrated Generation
Embedding& Retrieval
Incomplete Logical Chain
Sub-Q Contexts
3 Alternative Solution

QUESTION COMPLEXITY DETECTION

- The system first estimates whether a question requires multi-step reasoning.
- Metrics such as semantic scope and reasoning depth are used.
- If the question is complex it is decomposed into smaller reasoning units.
- This step ensures that retrieval operates on simpler logical sub-tasks.

$$\text{SplitCondition}(q) = \begin{cases} \text{True} & \text{if } \text{CL}(q) > \tau(t) \\ \text{False} & \text{otherwise} \end{cases} \quad (1)$$

where the *Cognitive Load* metric comprises:

$$\text{CL}(q) = \underbrace{\sigma(\text{Var}(\phi(q)))}_{\text{Semantic Scope}} + \underbrace{\sigma(\text{Depth}(q))}_{\text{Reasoning Steps}} + \underbrace{\sigma(\mathcal{H}(q))}_{\text{Ambiguity}}$$

QUESTION DECOMPOSITION

- Complex questions are split into atomic sub-questions.
- Each sub-question corresponds to one reasoning step.
- The answers to earlier steps become inputs to later steps.
- This creates a structured reasoning chain similar to multi-hop inference.

LOGIC-GUIDED RETRIEVAL

- Instead of retrieving documents once, retrieval happens at each reasoning step.
- The retrieval query incorporates the current sub-question and previous answers.
- This progressively refines the search context.
- As reasoning progresses, retrieval becomes increasingly targeted.

$$\mathbf{q}^{(i+1)} = \phi(\mathsf{concat}(a_i, q_{i+1})),$$

ATOMIC MEMORY BANK

- The system stores solved atomic questions and their answers.
- This allows reuse of intermediate results.
- It reduces redundant reasoning and improves stability.
- It also provides a transparent reasoning trace.

$$\max_{q', q'_i} \frac{q' \cdot q'_i}{\|q'\| \|q'_i\|} > \gamma$$

TERMINATION CONDITIONS

- The reasoning process proceeds through a sequence of atomic sub-questions. It terminates once the system has gathered sufficient information to answer the original question.
- Reasoning may also stop early if the system determines that additional retrieval steps are unlikely to produce useful information.
- This termination mechanism prevents the reasoning process from expanding indefinitely and avoids unnecessary retrieval or generation steps.
- By stopping when additional reasoning provides diminishing value, the system maintains computational efficiency while preserving answer quality.
- **Signals Used to Trigger Termination**
  - **Retriever confidence drop:** retrieval scores fall below a threshold, indicating that the system is no longer finding relevant supporting evidence.

$$\frac{1}{k}\sum_{i=1}^{k}\mathbb{I}\left[\mathrm{sim}(\mathbf{q}', c_i) < \delta\right] = 1,$$

  - **Dependency exhaustion:** all required sub-questions or logical dependencies have already been resolved.

$$\left(\bigwedge_{i=1}^{m}\mathrm{Answered}(q_i)\right) \wedge \neg\mathrm{Answerable}(q') \Rightarrow \mathrm{Stop}.$$

  - **Semantic saturation:** newly retrieved documents repeat previously obtained information and do not add new evidence.

LAG – KEY RESULTS

Method	HotpotQA		2Wiki		MuSiQue	
	Contain-Acc.	GPT-Acc.	Contain-Acc.	GPT-Acc.	Contain-Acc.	GPT-Acc.
Direct Zero-shot LLM Inference						
Llama3 (8B) (Meta, 2024)	23.7	20.1	33.8	15.4	6.4	6.0
GPT-3.5-turbo (OpenAI, 2024)	31.5	35.4	31.0	29.9	7.9	10.9
GPT-4o-mini (OpenAI, 2024)	30.4	34.2	29.0	28.6	7.8	10.1
Vanilla Retrieval-Augmented-Generation						
Retrieval (Top-3)	52.1	55.1	45.1	43.1	23.4	27.1
Retrieval (Top-5)	54.6	56.8	46.6	45.3	25.6	29.0
Retrieval (Top-10)	56.0	58.6	48.7	45.8	26.7	31.2
CoT (Top-5) (Wei et al., 2022)	55.1	57.1	48.7	45.9	27.1	30.7
IRCoT (Top-5) (Trivedi et al., 2023)	58.4	59.6	53.0	36.8	22.6	26.1
Novel Graph Retrieval-Augmented-Generation						
KGP (Wang et al., 2024)	56.2	57.1	52.2	33.9	30.5	27.3
G-retriever (He et al., 2024)	41.3	40.9	47.8	25.7	14.1	15.6
RAPTOR (Sarthi et al., 2024)	58.1	55.3	60.6	43.9	32.2	29.7
LightRAG (Guo et al., 2025)	61.5	60.5	54.4	38.0	27.7	28.3
HippoRAG (single-step) (Gutiérrez et al., 2024)	55.2	57.9	63.7	57.5	31.4	30.1
HippoRAG (multi-step) (Gutiérrez et al., 2024)	61.1	63.6	66.4	62.4	34.0	31.8
GFM-RAG (single-step) (Luo et al., 2025)	61.4	64.8	66.2	61.1	29.3	32.6
GFM-RAG (multi-step) (Luo et al., 2025)	63.4	65.5	69.5	63.2	31.5	35.5
HippoRAG 2 (Gutiérrez et al., 2025)	61.2	64.3	62.0	58.8	34.5	35.6
LinearRAG (Zhuang et al., 2025)	64.2	64.9	69.5	62.6	32.8	36.9
LAG(Ours)	**68.5**	**69.7**	**71.9**	**64.0**	**42.8**	**44.3**

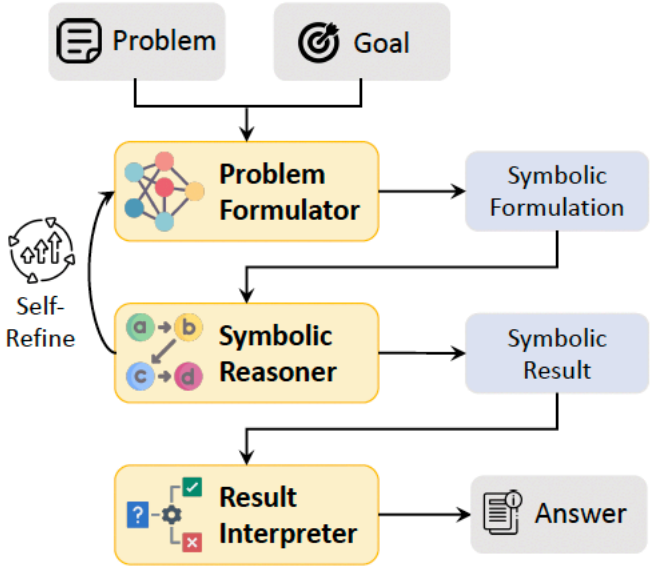
Method	GraphRAG-Bench	
	R Score	AR Score
KGP (Wang et al., 2024)	58.7	42.2
G-retriever (He et al., 2024)	60.2	43.7
LightRAG (Guo et al., 2025)	60.5	43.8
GFM-RAG (Luo et al., 2025)	60.4	44.3
HippoRAG (Gutiérrez et al., 2024)	60.9	44.6
HippoRAG 2 (Gutiérrez et al., 2025)	59.8	43.7
RAPTOR (Sarthi et al., 2024)	60.8	45.5
LinearRAG (Zhuang et al., 2025)	61.5	45.4
LAG(Ours)	**65.2**	**46.4**

- Evaluated on multi-hop QA datasets including HotpotQA and MuSiQue.
- LAG significantly outperforms standard RAG pipelines.
- Performance gains arise from better retrieval alignment with reasoning steps.
- The system improves both answer accuracy and reasoning quality.

SYMBOLIC-LOGIC SOLVER INTEGRATION

LOGIC-LM ARCHITECTURE

- **Three-stage reasoning pipeline:** The system decomposes logical reasoning into three explicit stages:
  - **1. Problem Formulation (LLM)**
  - Convert natural language into symbolic logic representation
  - identify entities, facts, and rules
  - generate logical formulas or constraints
  - Example output:
    - Fact: Bird(x) → Fly(x)
      Fact: Sparrow(Tweety)
      Query: Fly(Tweety)?
  - **2. Symbolic Reasoning (Solver)**
    - A deterministic solver performs inference on the logical representation.
    - Possible solvers used:
    - logic programming engines (forward/backward chaining)
    - first-order logic inference engines
    - constraint optimization solvers
    - SAT solvers
    - The solver computes the logically valid answer.
  - **3. Result Interpretation (LLM)**
    - The system converts the solver output into a final natural language answer.
      - map solver output → answer choice
      - optionally generate explanation

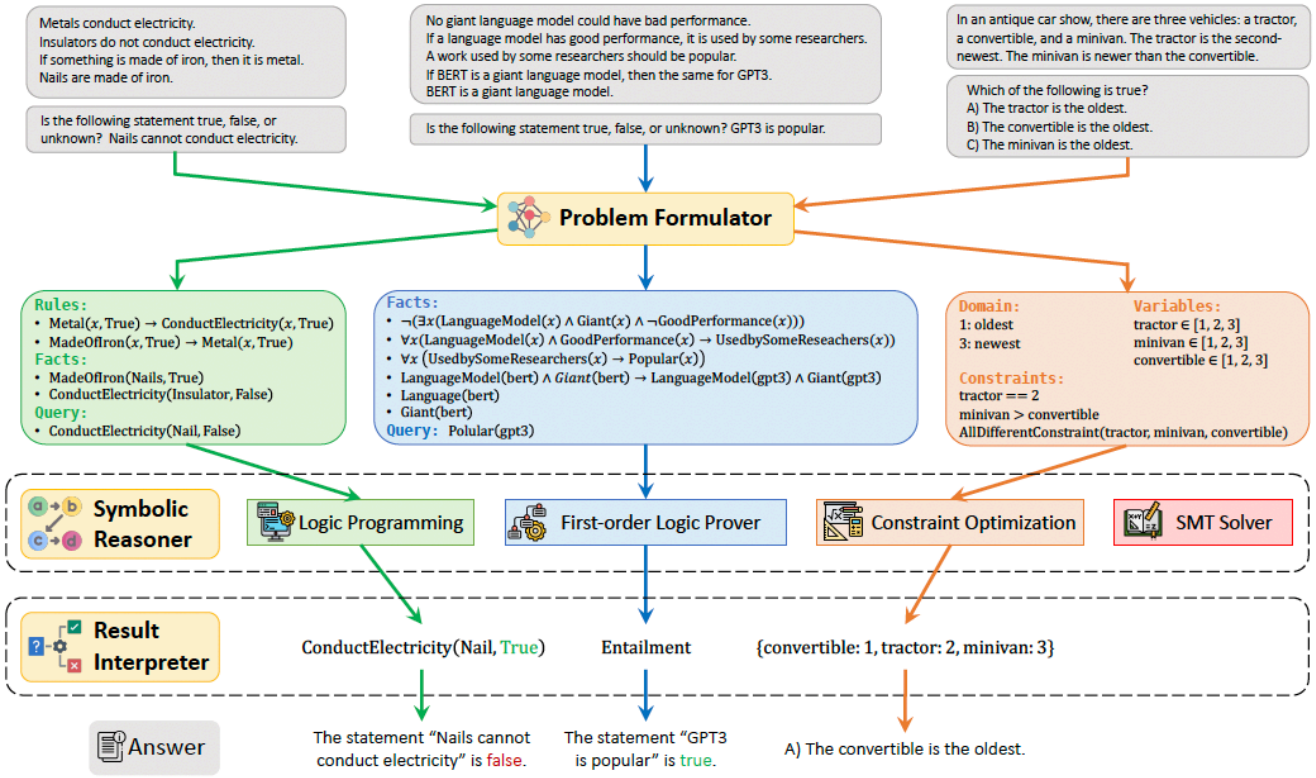


SELF-REFINEMENT

- **Major challenge: translating NL → symbolic logic**
  - Even strong LLMs often generate incorrect or unexecutable logical formulas
- Typical errors include:
  - incorrect logical relations
  - incorrect variable constraints
  - syntactically invalid formulas
- Example issues:
  - misinterpreting spatial relations (“above” vs “below”)
  - invalid FOL grammar expressions
- **Self-refinement mechanism**
  - If the symbolic solver fails:
    - solver returns an error message
    - the LLM analyzes the error
    - the LLM **revises the symbolic representation**
    - the solver is run again
  - This iterative loop improves the **executability and correctness of the logical formulation**.

THREE KEY MODULES

- **Problem Formulator:** generates symbolic representation for input problem (with LLMs via in-context learning)
- **Symbolic Reasoner :** logical inference on problem
- **Result Interpreter:** interpretation of symbolic answer

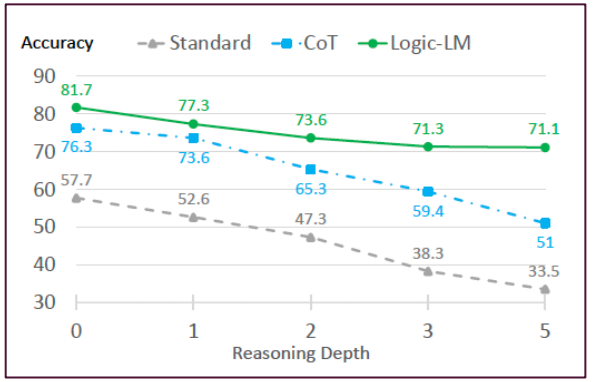


SOLVERS

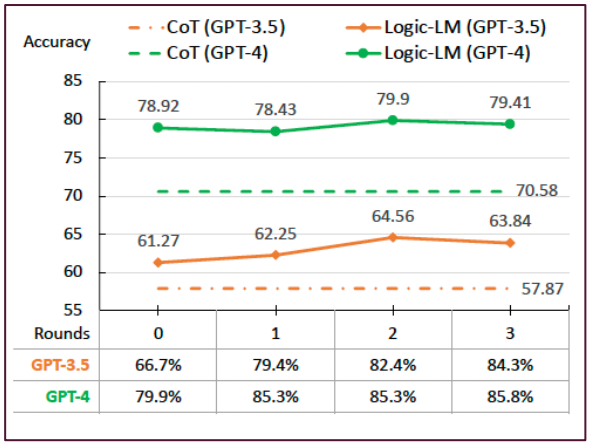
Solver Type	Best for	Example Problems
FOL Theorem Prover	logical deduction	knowledge reasoning, proofs
SMT Solver	logical + arithmetic constraints	LSAT puzzles, scheduling
SAT Solver	Boolean constraints	verification, configuration
Constraint Solver	combinatorial assignments	puzzles, planning

RESULTS

Dataset	ChatGPT (gpt-3.5-turbo)			GPT-3.5 (text-davinci-003)			GPT-4 (gpt-4)		
	Standard	CoT	Logic-LM	Standard	CoT	Logic-LM	Standard	CoT	Logic-LM
PrOntoQA	47.40	67.80	61.00	51.80	83.00	85.00	77.40	98.79	83.20
ProofWriter	35.50	49.17	58.33	36.16	48.33	71.45	52.67	68.11	79.66
FOLIO	45.09	57.35	62.74	54.60	57.84	61.27	69.11	70.58	78.92
LogicalDeduction	40.00	42.33	65.67	41.33	48.33	62.00	71.33	75.25	87.63
AR-LSAT	20.34	17.31	26.41	22.51	22.51	25.54	33.33	35.06	43.04



- **Performance improvements**
  - Logic-LM significantly outperforms pure LLM reasoning.
  - Key findings:
    - Average improvement of **≈39% over standard prompting**
    - Average improvement of **≈18% over Chain-of-Thought reasoning**
  - Example dataset improvements (GPT-3.5):
    - LogicalDeduction: **~40 → ~66 accuracy**
    - FOLIO: **~45 → ~63**
    - AR-LSAT: **~20 → ~26**
- Overall
  - Delegating inference to symbolic solvers improves: logical correctness, faithfulness of reasoning, performance on deep logical problems.
  - Symbolic solvers also handle **non-linear reasoning patterns** such as:
    - hypothesis testing
    - elimination
    - recursive inference (which pure CoT struggles with)



EXAMPLE, CODE REPO

```
Problem: "Stranger Things" is a popular Netflix show. If a Netflix show is popular, Karen will binge-watch it.
         If and only if Karen binge-watches a Netflix show, she will download it. Karen does not download "Black Mirror".
         "Black Mirror" is a Netflix show. If Karen binge-watches a Netflix show, she will share it to Lisa.
Question: Is the following statement true, false, or uncertain? "Black Mirror" is popular.
          (A) True               (B) False            (C) Uncertain
Generated symbolic Formulations:
    Predicates:
        NetflixShow(x) # x is a Netflix show.
        Popular(x) # x is popular.
        BingeWatch(x, y) # x binge-watches y.
        Download(x, y) # x downloads y.
        Share(x, y, z) # x shares y to z.
    Premises:
        NetflixShow(strangerThings) ∧ Popular(strangerThings) # "Stranger Things" is a popular Netflix show.
        ∀x (NetflixShow(x) ∧ Popular(x) → BingeWatch(karen, x)) # If a Netflix show is popular, Karen will binge-watch it.
        ∀x (NetflixShow(x) ∧ BingeWatch(karen, x) ↔ Download(karen, x)) # If and only if Karen … , she will download it.
        NetflixShow(blackMirror) ∧ ¬Download(karen, blackMirror) # Karen does not … "Black Mirror" is a Netflix show.
        ∀x (NetflixShow(x) ∧ BingeWatch(karen, x) → Share(karen, x, lisa)) # If Karen … , she will share it to Lisa.
    Conclusion:
        Popular(blackMirror) # "Black Mirror" is popular.
Predicted answer: B
```

- Code repo: https://github.com/teacherpeterpan/Logic-LLM
  - Please check this out
  - This is very relevant to CareTrace Final project (we will discuss more in the appropriate session)

VARIOUS KINDS OF LOGIC INTEGRATION: COMPARISON

Method	Where structure is introduced	What the LLM does	What external/ augmented components do
Chain-of-Thought	Prompting stage	Generates step-by-step reasoning text	No verification
Tree-of-Thought	Inference search	Proposes candidate reasoning steps	Tree search selects promising paths
Graph-of-Thought	Inference search	Generates reasoning fragments	Graph structure organizes and reuses them
Graph-of-Logic	Inference verification	Proposes logical inference steps	Logical rules verify correctness
LAG	Retrieval pipeline	Answers sub-questions	Decomposes question and controls retrieval
ALT (FLD_{x2})	Training data	Learns logical patterns	Synthetic dataset provides structured logic examples
Logic-LM	Inference solver	Translates NL → logical representation	Symbolic solver performs reasoning

IMPLICATIONS FOR NEUROSYMBOLIC AI

- Reliable reasoning rarely emerges from pure LLM generation alone; most successful systems introduce explicit structure around the LLM.
- Across recent work, structure is inserted at different stages of the pipeline:
  - training (ALT), inference/search (Tree/Graph-of-Thought, Graph-of-Logic), retrieval (LAG), or symbolic solving (Logic-LM).
- Consistent across all: LLMs generate candidate reasoning steps, while external mechanisms organize, constrain, or verify those steps.
- Making **intermediate reasoning states explicit** (steps, trees, graphs, logic programs) improves both accuracy and faithfulness of reasoning.
- The emerging paradigm: **hybrid reasoning systems**